

BFS og DFS i C# — puggeark

Tre traverseringer, én graf, én avgjørende forskjell · Teori på norsk, kode på engelsk

Grafen alt kjøres på

Alle tre eksemplene traverserer samme urettede graf, så output kan sammenlignes direkte:

```
      A
    /  \
   B    C
  /  \  /  \
 D    E  \  \
  \    \  \  \
   F---+ 
```

Adjacency list:

A: [B, C]	D: [B, C, F]
B: [A, D]	E: [C, F]
C: [A, D, E]	F: [D, E]

```
var graph = new Dictionary<char, List<char>>
{
    ['A'] = new() { 'B', 'C' },
    ['B'] = new() { 'A', 'D' },
    ['C'] = new() { 'A', 'D', 'E' },
    ['D'] = new() { 'B', 'C', 'F' },
    ['E'] = new() { 'C', 'F' },
    ['F'] = new() { 'D', 'E' }
};
```

Teorien du må eie først: LIFO mot FIFO

Hele forskjellen mellom BFS og DFS er **hvilken ende du tar neste node fra**:

FIFO (First In, First Out) — butikkøen. Den som stilte seg først, ekspederes først. Dette er `Queue<T>`: `Enqueue` legger bakerst, `Dequeue` tar forrest. Køen prioriterer *de eldste* oppdagelsene → grafen utforskes **nivå for nivå** → **BFS** (bredde-først).

LIFO (Last In, First Out) — tallerkenstabelen. Den du satte fra deg sist, tar du først. Dette er `Stack<T>`: `Push` legger øverst, `Pop` tar øverst. Stacken prioriterer *de nyeste* oppdagelsene → grafen utforskes **gren for gren, helt til bunns** → **DFS** (dybde-først).

Alt annet i koden — besøkt-settet, løkka, nabosjekken — er identisk. Kan du den ene, kan du begge.

To regler som gjelder uansett variant, og som skiller graf fra tre:

- Besøkt-settet er obligatorisk.** Grafer kan ha sykler (A-B-D-C-A er en ring her). Uten `visited` går traverseringen i evig runddans. Trær trengte det ikke — de har ingen sykler.
- Marker besøkt ved innlegging, ikke ved uttak.** Både B og C har D som nabo. Markeres D først når den tas ut, rekker begge å legge den inn — og den behandles to ganger.

Eksempel 1: BFS — iterativ med Queue

```
public static List<char> Bfs(Dictionary<char, List<char>> graph, char start)
{
    var order = new List<char>();
    var visited = new HashSet<char> { start };    // marked when ENQUEUED
    var queue = new Queue<char>();
    queue.Enqueue(start);

    while (queue.Count > 0)
    {
        var node = queue.Dequeue();                // take from the FRONT
        order.Add(node);

        foreach (var neighbor in graph[node].OrderBy(n => n))
        {
            if (visited.Contains(neighbor)) continue;
            visited.Add(neighbor);
            queue.Enqueue(neighbor);                // newcomers wait at the BACK
        }
    }
    return order;
}
```

Output: A B C D E F (nivåene: A | B,C | D,E | F)

Les rytmen: A tas ut og legger B, C bakerst. Neste `Dequeue` gir B — den som har *ventet lengst*. B sine naboer stiller seg bak C og må vente. Hele nivå 1 behandles før noe på nivå 2 slipper til. Derfor er første gang BFS når en node også korteste vei dit (i uvektet graf).

Puggeknagg BFS: «Kølapp på legevakten» — nye pasienter bakerst, eldste først. Ordparet som skal sitte i fingrene: `Queue / Enqueue / Dequeue`. BFS = korteste vei, nivåer, nærmeste X.

Eksempel 2: DFS — iterativ med Stack

Sammenlign linje for linje med BFS — **bare datastrukturen er byttet:**

```
public static List<char> DfsIterative(Dictionary<char, List<char>> graph, char start)
{
    var order = new List<char>();
    var visited = new HashSet<char> { start };
    var stack = new Stack<char>();                // the ONLY change
    stack.Push(start);

    while (stack.Count > 0)
    {
        var node = stack.Pop();                    // take from the TOP
        order.Add(node);

        foreach (var neighbor in graph[node].OrderBy(n => n))
        {
            if (visited.Contains(neighbor)) continue;
            visited.Add(neighbor);
            stack.Push(neighbor);                    // newcomers land ON TOP
        }
    }
    return order;
}
```

Output: A C E F D B (stuper: A → C → E → F → D, backer, B til slutt)

Les rytmen: A pusher B så C — C ligger øverst. `Pop` gir C, den *nyeste* oppdagelsen. C sine naboer legges oppå, og en av dem tas straks. Hvert steg graver dypere i det siste funnet. B — oppdaget aller først — synker til bunns og behandles sist.

Puggeknagg DFS: «Tallerkenstabelen» — sist inn, først ut. Ordparet: `Stack / Push / Pop`. DFS = sykelsjekk, finnes-det-en-vei, topologisk sortering.

Eksempel 3: DFS — rekursiv (kall-stacken gjør jobben)

```
public static List<char> DfsRecursive(Dictionary<char, List<char>> graph, char start)
{
    var order = new List<char>();
    var visited = new HashSet<char>();
    Dfs(start);
    return order;

    void Dfs(char node) // local helper function
    {
        visited.Add(node);
        order.Add(node);

        foreach (var neighbor in graph[node].OrderBy(n => n))
            if (!visited.Contains(neighbor))
                Dfs(neighbor); // the call stack IS the stack
    }
}
```

Output: A B D C E F

Hvorfor rekursjon passer DFS: rekursjon *er* en stack. Hvert kall legges øverst på kall-stacken, retur tar av øverst — LIFO er innebygd i selve mekanismen. Kall-stacken husker veien du gikk og hvor du skal fortsette når du backer. Dette er samme mønster som `Inorder` på binærtre — DFS på tre og DFS på graf er samme algoritme, grafen trenger bare `visited` i tillegg.

Hvorfor de to DFS-variantene gir ulik rekkefølge: den rekursive tar naboene direkte i alfabetisk rekkefølge (B før C). Den iterative pusher B så C — og popper dermed C først, i *omvendt* rekkefølge av innleggingen. Begge er gyldige DFS: begge følger en gren helt til bunns før neste påbegynnes.

Og rekursiv BFS? Finnes ikke i noen naturlig form. Rekursjon gir LIFO gratis; BFS trenger FIFO. Enhver «rekursiv BFS» må smugle inn en kø ved siden av — og da gjør rekursjonen ingen jobb, den koster bare stackrammer (og risikerer `StackOverflowException`, siden C# ikke har tail call-optimalisering). Regel: **DFS finnes i to likeverdige former, BFS i én.** Skriv alltid BFS med `Queue` og `while`.

Alt på ett brett

	BFS	DFS iterativ	DFS rekursiv
Datastruktur	Queue<T> (FIFO)	Stack<T> (LIFO)	kall-stacken (LIFO)
Ordpar	Enqueue / Dequeue	Push / Pop	kall / retur
Utforsker	nivå for nivå	gren for gren	gren for gren
Output her	A B C D E F	A C E F D B	A B D C E F
Garanterer	korteste vei (uvektet)	ingenting om lengde	ingenting om lengde
Typisk bruk	korteste vei, nærmeste X, nivåer	når rekursjon er uaktuelt (dype grafer)	sykelsjekk, sammenheng, topologisk sortering
Minne	O(bredeste nivå)	O(dypeste gren)	O(dypeste gren) + fare for stack overflow
Tid	O(V + E) for alle tre — hver node og kant røres én gang		

Slik pugger du til det sitter

- Skriv BFS blindt.** Blankt ark eller tom fil, ingen kiking. Kjør mot grafen — får du `A B C D E F`, sitter den.
- Lag DFS iterativ av den** ved kun å bytte datastruktur og de to metodenavnene. Sjekk: `A C E F D B`. Denne øvelsen brenner inn at forskjellen er én linje.
- Skriv DFS rekursiv blindt.** Sjekk: `A B D C E F`. Kjenn igjen `Inorder`-mønsteret: besøk, så rekursjon på hver ubesøkt nabo.
- Legg inn `Console.WriteLine(string.Join(" ", queue))`** (eller stacken) øverst i `while`-løkka og se de to rytmene: køen fylles jevnt bakfra, stacken pulserer opp og ned.
- Forklar høyt** — uten notater — hvorfor BFS gir korteste vei og hvorfor rekursiv BFS ikke finnes. Kan du begrunne, ikke bare gjengi, sitter det for godt.
- Gjenta etter 2 dager og etter en uke.** Spredt repetisjon slår skippertak — det er få nok linjer til at fem minutter holder per runde.

De tre vanligste feilene (sjekk deg selv mot disse når du skriver blindt):

- Glemt `visited` → evig løkke i graf med sykkel.
- Markerer besøkt ved *uttak* i stedet for innlegging → samme node havner flere ganger i køen/stacken.
- Glemt å legge `start` i både `visited` og køen/stacken før løkka → enten dobbeltbesøk av start eller tom løkke.